

Dispensador inteligente de comida para gatos

Ing. Durante Matías Nahuel

Carrera de Especialización en Sistemas Embebidos

Director: Esp. Ing. Santiago Bualó (FIUBA)

Jurados:

Esp. Ing. Martin Duarte (FIUBA)
Esp. Ing. Santiago Escibá (FIUBA)
Esp. Ing. Lucas Monzón Languasco (FIUBA)

Ciudad de Córdoba, mayo de 2026

Resumen

La presente memoria describe el trabajo realizado para el desarrollo de un sistema inteligente de dispensado de comida para gatos. La finalidad del trabajo es mejorar el bienestar animal mediante la automatización del suministro de alimento y el monitoreo del consumo a través de un software. Para ello se emplearon un microcontrolador de alto desempeño, sensores de pesaje y módulos de comunicación inalámbrica, lo que permitió obtener una solución que combina el cuidado de mascotas con los avances de los sistemas embebidos y la Internet de las Cosas (IoT).

El desarrollo abarcó el diseño del hardware, la implementación del software de control y la validación del sistema mediante pruebas funcionales. Se aplicaron conocimientos en electrónica, programación de microcontroladores y protocolos de comunicación, con el fin de obtener un producto confiable, autónomo y con potencial de aplicación comercial.

Agradecimientos

A mi novia, mi familia, amigos y compañeros.

A mi director de trabajo, Esp. Ing. Santiago Bualó, por compartir sus conocimientos y orientación a lo largo de la especialización.

A mis colegas ingenieros, por el apoyo constante, los valiosos consejos y las sugerencias técnicas brindadas durante cada etapa del desarrollo de este trabajo.

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción a los sistemas de alimentación automatizada	1
1.1.1. Alimentación y bienestar animal	1
1.1.2. Domótica aplicada al cuidado de mascotas	1
1.2. Contexto y motivación	2
1.3. Estado del arte	2
1.3.1. PetSafe Smart Feed	2
1.3.2. PETLIBRO Granary Wi-Fi	3
1.3.3. Análisis del estado del arte y aportes del desarrollo	4
1.4. Objetivos del trabajo	5
2. Introducción específica	7
2.1. Componentes de hardware del sistema	7
2.1.1. Microcontrolador STM32F446RE	7
2.1.2. Módulo ESP32	8
2.1.3. Celda de carga y módulo HX711	8
2.1.4. Motor paso a paso y driver A4988	9
2.1.5. Módulo RTC DS3231	9
2.2. Software y entorno de desarrollo	10
2.2.1. STM32CubeIDE	10
2.2.2. FreeRTOS	10
2.2.3. Doxygen	10
2.2.4. PlatformIO	10
2.2.5. Flutter	10
2.2.6. Lenguaje de programación C	11
2.3. Protocolos de comunicación	11
2.3.1. Protocolo I2C	11
2.3.2. Protocolo UART	11
2.3.3. Bluetooth Classic	11
3. Diseño e implementación	13
3.1. Arquitectura general del sistema	13
3.2. Arquitectura de hardware	14
3.2.1. Gestión de alimentación	14
3.2.2. Interconexión de componentes	14
3.2.3. Análisis de potencia	16
3.3. Diseño del firmware	16
3.3.1. Organización del firmware	16
3.3.2. Módulo orquestador	17
3.3.3. Módulo de periféricos	17

3.3.4. Módulo de pesaje	18
3.3.5. Módulo de gestión de horarios	18
3.3.6. Módulo de gestión de almacenamiento	18
3.4. Comunicaciones e interfaces	19
3.4.1. Módulo de comunicación	19
3.4.2. Interfaz de usuario	19
3.5. Integración hardware-software	20
4. Ensayos y resultados	23
4.1. Pruebas funcionales del hardware	23
5. Conclusiones	25
5.1. Conclusiones generales	25
5.2. Próximos pasos	25
Bibliografía	27

Índice de figuras

1.1. Dispensador automático PetSafe <i>Smart Feed</i> . ¹	3
1.2. Dispensador automático PETLIBRO <i>Granary Smart Feeder</i> . ²	4
2.1. Placa de evaluación Nucleo de STM. ³	7
2.2. Módulo ESP32. ⁴	8
2.3. Celda de carga y módulo HX711. ⁵	8
2.4. Motor paso a paso NEMA 17 y <i>driver</i> A4988. ⁶	9
2.5. Módulo RTC DS3231. ⁷	9
3.1. Diagrama en bloques del sistema.	13
3.2. Interconexión entre la fuente de alimentación y el microcontrolador.	14
3.3. Interconexión entre el microcontrolador y sus periféricos.	15
3.4. Interconexión entre el microcontrolador, el driver A4988 y el motor NEMA 17.	15
3.5. Diagrama en bloques de los subsistemas de software.	16
3.6. Diagrama de estados del dispensador.	17
3.7. Arquitectura de comunicación del sistema de dispensado.	19
3.8. Diagrama de navegación de la aplicación de escritorio.	20
3.9. Diagrama de integración hardware-software del sistema.	20

Índice de tablas

1.1. Comparación de características entre productos del estado del arte y el sistema desarrollado	5
3.1. Consumo máximo de cada componente del sistema.	16

A mi novia Sofía, por su compañía durante la cursada de la especialización y su apoyo constante durante el desarrollo del trabajo.

A mi familia, por el amor y el orgullo que sienten en cada pequeño paso que doy en la dirección de lo que elegí para mi vida: la electrónica.

A todos ellos, ¡muchas gracias!

Capítulo 1

Introducción general

En este capítulo se presentan los conceptos fundamentales relacionados con los sistemas de alimentación automatizada para mascotas, el análisis del estado del arte, la motivación técnica y los objetivos que rigen el presente trabajo.

1.1. Introducción a los sistemas de alimentación automatizada

El cuidado de mascotas en el hogar ha evolucionado significativamente gracias al avance de los sistemas embebidos y la electrónica de bajo costo. En particular, la alimentación de animales domésticos representa una de las principales preocupaciones de los propietarios, especialmente ante jornadas laborales extensas o ausencias prolongadas del hogar que dificultan el cumplimiento de rutinas nutricionales estrictas. La automatización de este proceso permite asegurar que los animales reciban la cantidad de alimento adecuada en los horarios correctos, independientemente de la presencia de sus dueños.

1.1.1. Alimentación y bienestar animal

Uno de los principales desafíos en el cuidado de gatos domésticos es garantizar una alimentación regular y controlada. Una rutina alimenticia inadecuada puede derivar en patologías frecuentes como la obesidad felina, la diabetes y los trastornos digestivos [1], todas ellas asociadas a variaciones en la cantidad o los horarios de ingesta.

La medicina veterinaria establece que la ración diaria debe determinarse en función del peso, la edad y el estado de salud del animal, y distribuirse en momentos fijos del día para evitar ansiedad y alteraciones del comportamiento [2]. Conocer si el animal efectivamente consumió el alimento dispensado agrega una dimensión diagnóstica que los sistemas de alimentación tradicionales no contemplan.

1.1.2. Domótica aplicada al cuidado de mascotas

La domótica, entendida como la aplicación de tecnologías de automatización y control al entorno doméstico, ha experimentado un crecimiento sostenido en los últimos años [3]. La disponibilidad de microcontroladores de bajo costo, módulos de comunicación inalámbrica y sensores de alta precisión ha permitido extender estas soluciones a ámbitos que antes resultaban inaccesibles para el usuario común.

En el área específica del cuidado de mascotas, esta tendencia se traduce en dispositivos capaces de automatizar tareas rutinarias como la alimentación, la hidratación y el monitoreo de actividad, integrándose en muchos casos a plataformas de Internet de las Cosas (IoT) que permiten el acceso remoto desde dispositivos móviles.

1.2. Contexto y motivación

La motivación principal de este trabajo surge de la necesidad de contar con un sistema de alimentación para gatos que no solo automatice la entrega de comida, sino que también brinde información confiable sobre el comportamiento alimenticio de la mascota. Esta información resulta valiosa tanto para el propietario como para el veterinario, ya que permite detectar anomalías en el consumo que podrían indicar alteraciones en el estado de salud del animal.

Por otra parte, la disponibilidad de microcontroladores de alta capacidad, módulos de comunicación inalámbrica y sensores de precisión a precios accesibles representa una oportunidad para desarrollar soluciones tecnológicas que superen las limitaciones de los productos comerciales existentes. La Carrera de Especialización en Sistemas Embebidos de la FIUBA brindó las herramientas conceptuales y prácticas necesarias para llevar adelante un desarrollo de este tipo con rigor técnico y metodológico.

1.3. Estado del arte

En esta sección se presentan algunos productos disponibles en el mercado que abordan problemáticas similares a la planteada en este trabajo. Se realiza un análisis de sus características técnicas, limitaciones y diferencias respecto de la solución desarrollada.

1.3.1. PetSafe Smart Feed

PetSafe Smart Feed [4] es uno de los dispensadores automáticos para mascotas más difundidos en el mercado internacional. El dispositivo cuenta con conectividad Wi-Fi de 2,4 GHz y se gestiona mediante la aplicación móvil My PetSafe [5], compatible con iOS y Android. Permite programar hasta doce raciones diarias con tamaños que van desde 1/8 de taza (29 ml) hasta 4 tazas (946 ml) por comida, y dispone de una capacidad de almacenamiento de 24 tazas (5 678 ml) de alimento seco o semihúmedo. La aplicación envía notificaciones ante nivel bajo de alimento, depósito vacío o pérdida de conectividad. La figura 1.1 muestra el dispositivo, que se comercializa a un precio de USD 120,69.



FIGURA 1.1. Dispensador automático PetSafe *Smart Feed*.¹

Sin embargo, el sistema no cuenta con ningún mecanismo de pesaje del plato, por lo que no es posible determinar si el animal consumió efectivamente el alimento entregado. El historial disponible en la aplicación refleja únicamente los eventos de dispensado, y el funcionamiento depende de manera permanente de una conexión a Internet.

1.3.2. PETLIBRO Granary Wi-Fi

El modelo PETLIBRO Granary [6] con conectividad Wi-Fi es un dispensador automático de alimento seco para gatos y mascotas de tamaño pequeño o mediano. Soporta redes de 2,4 GHz y 5 GHz, y permite configurar hasta diez comidas diarias con porciones ajustables de 20 ml, sobre una capacidad total de 5 litros. Su mecanismo de dispensado por paletas de distribución uniforme reduce la ocurrencia de atascos, y el sistema incorpora sellado cuádruple con bolsa desecante para preservar la frescura del alimento. Cuenta con batería de respaldo ante cortes de energía. La figura 1.2 muestra el dispositivo, que se comercializa a un precio de USD 89,99 en el mercado estadounidense.

¹<https://www.petsafe.com/es/p/alimentador-de-mascotas-automatico-e-inteligente-smart-feed/PFD19-16861/>



FIGURA 1.2. Dispensador automático PETLIBRO *Granary Smart Feeder*.²

Al igual que el producto anterior, no incorpora un sistema de monitoreo del consumo real. El historial disponible en la aplicación registra únicamente los eventos de dispensado, sin ningún sensor que permita verificar si el animal ingirió efectivamente la ración entregada.

1.3.3. Análisis del estado del arte y aportes del desarrollo

Del análisis de los productos presentados se concluye que, si bien existen soluciones que automatizan la entrega de alimento con conectividad inalámbrica y control remoto, ninguna contempla la medición directa del consumo por parte del animal. Esta limitación impide obtener información confiable sobre el comportamiento alimenticio de la mascota, que es el dato de mayor valor para la detección temprana de alteraciones en su estado de salud. Adicionalmente, ambos productos analizados dependen de manera permanente de una conexión a Internet para operar, lo que los hace vulnerables ante cortes de conectividad o energía.

El sistema desarrollado en este trabajo incorpora como principal diferencial un módulo de pesaje del plato basado en una celda de carga HX711 [7], que permite determinar con precisión si el alimento dispensado fue consumido. La gestión autónoma de horarios se logra mediante un reloj de tiempo real (RTC), lo que elimina la dependencia de Internet. La comunicación con el usuario se realiza a través de Bluetooth con una aplicación en PC, y la gestión eficiente de tareas concurrentes se implementa sobre FreeRTOS [8]. La tabla 1.1 presenta una comparación de las características principales entre los productos analizados y el sistema desarrollado.

²<https://petlibro.com/products/petlibro-5g-wifi-automatic-pet-feeder>

TABLA 1.1. Comparación de características entre productos del estado del arte y el sistema desarrollado.

Característica	PetSafe Smart Feed	PETLIBRO Granary	Sistema desarrollado
Conectividad	Wi-Fi 2,4 GHz	Wi-Fi 2,4/5 GHz	Bluetooth
Raciones programables	Hasta 12/día	Hasta 10/día	Por horario (RTC)
Pesaje del plato	No	No	Sí
Dependencia de Internet	Permanente	Permanente	No requerida
Batería de respaldo	No	Sí	Sí
Dispensado manual	No informado	No informado	Sí (pulsador físico)
Precio aproximado	USD 120,69	USD 89,99	Bajo costo

Entre los aportes descritos, el módulo de pesaje del plato representa el diferencial más significativo, ya que ninguno de los productos comerciales analizados es capaz de determinar si el animal consumió efectivamente el alimento entregado. La operación autónoma ante cortes de conectividad y el bajo costo de implementación con componentes disponibles en el mercado local complementan esta ventaja, configurando una solución con potencial de aplicación tanto doméstica como clínica.

1.4. Objetivos del trabajo

Este trabajo tuvo como objetivo desarrollar un prototipo funcional de un dispensador inteligente de alimento seco para gatos, basado en un microcontrolador STM32F446RE [9], que permita automatizar la entrega de comida en horarios programados, monitorear el consumo mediante el pesaje del plato y comunicar la información al usuario a través de Bluetooth.

El trabajo integró los conocimientos adquiridos en la Carrera de Especialización en Sistemas Embebidos de la FIUBA y abarcó el diseño del hardware, la implementación del firmware en lenguaje C sobre FreeRTOS y la validación del sistema mediante pruebas funcionales.

Por otra parte, no forma parte de este trabajo el desarrollo de una aplicación móvil completa ni de una plataforma web. Tampoco se contempla el diseño definitivo de la estructura mecánica del dispositivo, ni su adaptación para uso comercial.

Capítulo 2

Introducción específica

En este capítulo se describen los componentes de hardware, el software y los protocolos de comunicación utilizados para el desarrollo del sistema.

2.1. Componentes de hardware del sistema

En esta sección se describen los componentes electrónicos y módulos que conforman el hardware del sistema, entre ellos el microcontrolador, el motor de dispensado, la celda de carga y los módulos de comunicación y temporización.

2.1.1. Microcontrolador STM32F446RE

El microcontrolador empleado es el modelo STM32F446RE de STMicroelectronics, que actúa como unidad central de procesamiento del sistema y gestiona el dispensado de alimento, la lectura de sensores y la comunicación con los módulos periféricos. Está basado en el núcleo ARM Cortex-M4 con una frecuencia de trabajo de hasta 180 MHz, dispone de una unidad de punto flotante de precisión simple y memorias embebidas de alta velocidad, entre ellas 512 kB de memoria flash y 128 kB de SRAM.

Cuenta con múltiples entradas y salidas digitales y analógicas, convertidores analógicos-digitales de 12 bits y hasta 17 temporizadores. Posee además diversas interfaces de comunicación, entre las que se destacan UART [10] (*Universal Asynchronous Receiver-Transmitter*) e I²C [11] (*Inter-Integrated Circuit*). Para el desarrollo de este trabajo se utilizó la placa de evaluación Nucleo-F446RE [12], que puede observarse en la figura 2.1, y que integra el módulo ST-LINK necesario para la programación y depuración del microcontrolador.

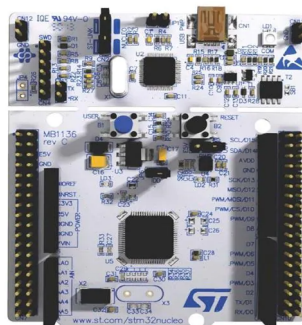


FIGURA 2.1. Placa de evaluación Nucleo de STM.¹

¹<https://www.st.com/en/evaluation-tools/nucleo-f446re.html>

2.1.2. Módulo ESP32

Para la comunicación inalámbrica con el usuario se utilizó un módulo ESP32, que actuó como puente (bridge) entre el microcontrolador STM32 y la aplicación en PC. La comunicación entre el STM32 y el ESP32 se realizó a través del protocolo UART, mientras que el ESP32 gestionó la comunicación Bluetooth con la aplicación receptora.

En cuanto a sus características técnicas, el ESP32 es un SoC (*System on Chip*) desarrollado por Espressif Systems [13]. Cuenta con doble núcleo Xtensa LX6 a 240 MHz e integra conectividad Wi-Fi 802.11b/g/n y Bluetooth 4.2, con soporte para Bluetooth Classic y BLE (*Bluetooth Low Energy*). En la figura 2.2 se muestra el módulo, donde se distingue la antena Bluetooth integrada en el extremo superior. En este trabajo se utilizó exclusivamente la funcionalidad Bluetooth Classic para la transmisión de datos en formato JSON (*JavaScript Object Notation*).



FIGURA 2.2. Módulo ESP32.²

2.1.3. Celda de carga y módulo HX711

Para medir el peso del alimento en el plato se utilizó una celda de carga tipo galga extensiométrica combinada con el módulo amplificador y convertor analógico-digital HX711. La celda de carga convierte la fuerza aplicada en una variación de resistencia eléctrica de muy baja amplitud. Dado que el microcontrolador no puede procesar señales analógicas de esa magnitud directamente, se incorporó el módulo HX711 que amplifica y digitaliza dicha señal con una resolución de 24 bits. En la figura 2.3 se ilustran la celda de carga en la parte superior y el módulo HX711 con sus terminales de conexión en la parte inferior.

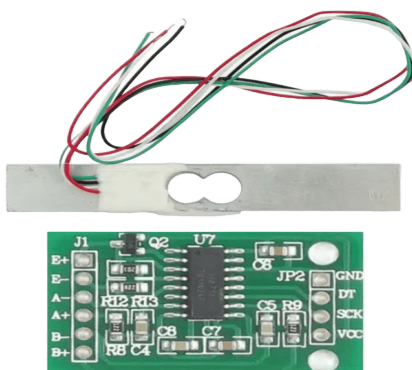


FIGURA 2.3. Celda de carga y módulo HX711.³

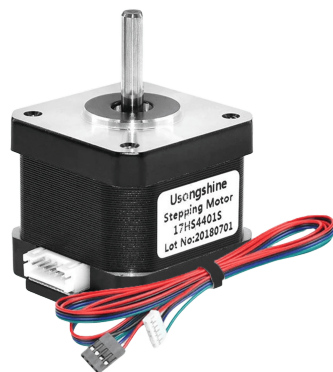
²https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

³https://cdn.sparkfun.com/assets/b/f/5/a/e/hx711F_EN.pdf

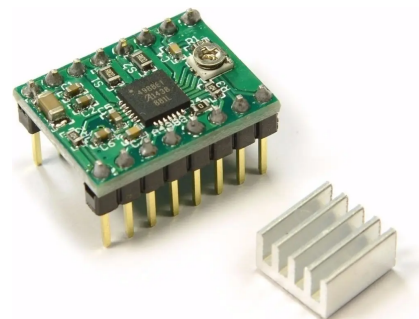
Para la comunicación con el microcontrolador, el HX711 utiliza una interfaz de dos cables (CLK y DATA) y opera con una tensión de alimentación de 5 V o 3,3 V. Su tasa de muestreo configurable es de 10 o 80 muestras por segundo, lo que resulta suficiente para la aplicación de monitoreo de consumo en intervalos regulares.

2.1.4. Motor paso a paso y driver A4988

El mecanismo de dispensado de alimento fue accionado por un motor paso a paso NEMA 17 [14], seleccionado por su precisión de movimiento y su par motor. Para su control se utilizó el driver A4988 de Pololu [15], un módulo compacto que permite gestionar motores paso a paso bipolares con corrientes de hasta 2 A por fase. En la figura 2.4 se muestran el motor NEMA 17 (figura 2.4a) y el driver A4988 junto con su disipador térmico (figura 2.4b).



(A) Motor paso a paso NEMA 17.



(B) Driver A4988.

FIGURA 2.4. Motor paso a paso NEMA 17 y driver A4988.⁴

El driver A4988 acepta señales de dirección (DIR) y paso (STEP) provenientes del microcontrolador y permite ajustar la resolución de movimiento mediante micropasos. Su tensión de alimentación lógica es de 3,3 V a 5,5 V, mientras que la tensión de potencia para el control del motor puede ser de hasta 35 V.

2.1.5. Módulo RTC DS3231

Para la gestión autónoma de los horarios de dispensado se incorporó un módulo de reloj de tiempo real (RTC) basado en el integrado DS3231 de Maxim Integrated [16], que puede observarse en la figura 2.5. Este dispositivo proporciona información de fecha y hora con una precisión de ± 2 ppm, equivalente a aproximadamente 1 minuto de desviación por año.

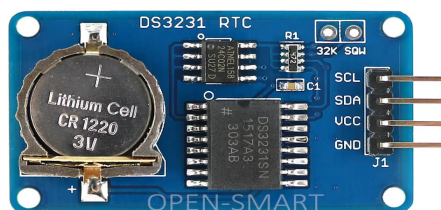


FIGURA 2.5. Módulo RTC DS3231.⁵

⁴<https://www.pololu.com/product/1182>

⁵<https://www.analog.com/en/products/ds3231.html>

El DS3231 se comunica con el microcontrolador mediante el protocolo I²C y dispone de una batería de respaldo (batería de litio tipo botón CR1220) que mantiene el funcionamiento del reloj ante cortes de energía. De este modo, los horarios de dispensado programados se conservan aun cuando el sistema se encuentre sin alimentación principal.

2.2. Software y entorno de desarrollo

En esta sección se describen las herramientas de software y los entornos de desarrollo utilizados para la implementación del sistema, tanto para el firmware del microcontrolador como para el módulo inalámbrico y la aplicación de escritorio.

2.2.1. STM32CubeIDE

STM32CubeIDE [17] es un entorno de desarrollo integrado (IDE) de código abierto utilizado para la programación del firmware del microcontrolador STM32F446RE. Basado en Eclipse, permite configurar periféricos, generar código inicial de forma automática, compilar y depurar en la misma plataforma.

2.2.2. FreeRTOS

FreeRTOS es un sistema operativo de tiempo real (RTOS) de código abierto ampliamente utilizado en sistemas embebidos. Provee mecanismos de planificación de tareas, semáforos, colas de mensajes y temporizadores de software que facilitan la gestión de múltiples procesos concurrentes. En este trabajo, FreeRTOS se integró como middleware sobre la capa de abstracción de hardware (HAL) del STM32 para gestionar las tareas de lectura de sensores, control del motor, comunicación UART y manejo del pulsador.

2.2.3. Doxygen

Doxygen [18] es una herramienta de código abierto para la generación automática de documentación a partir de comentarios estructurados en el código fuente. En este trabajo se utilizó para documentar el firmware del STM32 y obtener una referencia técnica en formato HTML (*HyperText Markup Language*) que describe las funciones, parámetros y estructuras de datos implementadas.

2.2.4. PlatformIO

Para el desarrollo del firmware del módulo ESP32 se utilizó PlatformIO [19], una extensión del editor Visual Studio Code que provee soporte para múltiples plataformas de desarrollo embebido. Este entorno simplifica el acceso a las bibliotecas de comunicación Bluetooth Classic (SPP, *Serial Port Profile*).

2.2.5. Flutter

Flutter [20] es un framework de código abierto desarrollado por Google para la creación de aplicaciones multiplataforma a partir de una única base de código. En este trabajo se utilizó para desarrollar la aplicación de escritorio que corre en Windows y se comunica con el módulo ESP32 mediante Bluetooth Classic. Esta aplicación permite al usuario visualizar el historial de consumo y configurar los horarios de dispensado.

2.2.6. Lenguaje de programación C

El firmware del microcontrolador STM32 se desarrolló íntegramente en lenguaje C, un lenguaje de propósito general que permite el control directo del hardware y la gestión eficiente de los recursos del sistema. Su portabilidad y compatibilidad con las bibliotecas HAL de STM32 y con FreeRTOS lo convirtieron en la opción más adecuada para este tipo de desarrollo.

2.3. Protocolos de comunicación

En esta sección se describen los tres protocolos de comunicación empleados para la transferencia de datos entre los distintos módulos del sistema. La elección de cada protocolo respondió a los requisitos de interfaz de cada módulo y a decisiones de diseño del sistema.

2.3.1. Protocolo I2C

Este protocolo consiste en un bus serie síncrono, bidireccional y semidúplex, organizado bajo una arquitectura maestro-esclavo. Utiliza dos líneas eléctricas: SCL para la señal de reloj y SDA para la transferencia de datos. La comunicación se basa en tramas que incluyen la dirección de 7 bits del dispositivo, un bit de lectura/escritura y un mecanismo de confirmación mediante señales ACK (*Acknowledgement*) y NACK (*Not Acknowledgement*) [11].

2.3.2. Protocolo UART

El protocolo UART se utilizó para la comunicación serie entre el microcontrolador STM32 y el módulo ESP32. Este protocolo es asíncrono y utiliza dos líneas físicas: una de transmisión (TX) y otra de recepción (RX). Los datos se transmiten en tramas compuestas por un bit de inicio, los bits de datos (generalmente 8) y un bit de parada. En este sistema se configuraron ambos extremos con una velocidad de 115200 baudios, sin paridad y con un bit de parada [10].

2.3.3. Bluetooth Classic

El módulo ESP32 gestionó la comunicación inalámbrica con la aplicación en PC mediante Bluetooth Classic bajo el perfil SPP (*Serial Port Profile*). Este perfil emula una comunicación serie sobre el canal Bluetooth, lo que facilita la integración con el microcontrolador, ya que los datos recibidos por UART desde el STM32 fueron reenviados por Bluetooth, y viceversa. Los datos se transmiten en formato JSON, lo que permite estructurar la información de manera legible y extensible [21].

Capítulo 3

Diseño e implementación

En este capítulo se describen el diseño e implementación del sistema desarrollado, tanto del dispositivo en general como los diferentes módulos de firmware que lo componen. También se presenta la interfaz de usuario y el protocolo de comunicación empleado entre el microcontrolador y la aplicación de escritorio.

3.1. Arquitectura general del sistema

El sistema desarrollado constituye un dispensador automático de alimento seco para gatos, capaz de operar de forma autónoma sin dependencia de una conexión a Internet. El dispositivo gestiona la entrega de alimento en horarios programados y monitorea el peso del tanque de almacenamiento y del plato de consumo. Esta información se comunica al usuario a través de una aplicación de escritorio mediante Bluetooth.

El componente central del sistema es el microcontrolador STM32F446RE, que actúa como unidad de procesamiento principal. Este componente coordina la lectura de los sensores de pesaje, controla el motor de dispensado y administra la comunicación con los módulos periféricos. La figura 3.1 presenta el diagrama en bloques del sistema, donde se distinguen tres bloques funcionales principales: sensado, control y comunicación con el usuario.

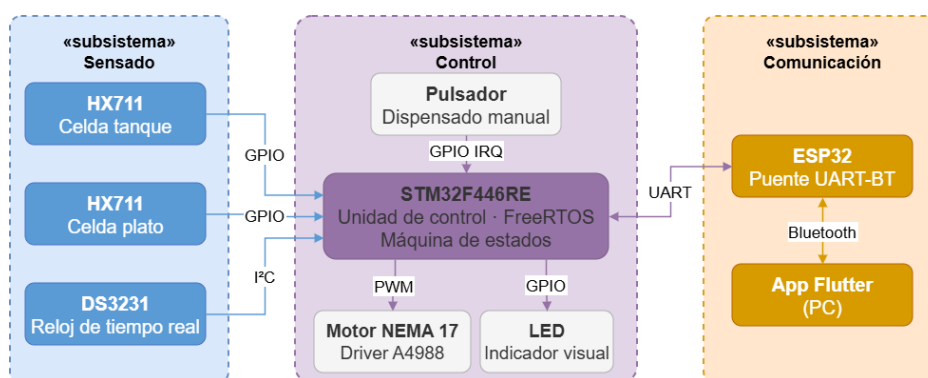


FIGURA 3.1. Diagrama en bloques del sistema.

El bloque de sensado comprende dos módulos HX711 conectados a sus respectivas celdas de carga: uno ubicado bajo el tanque de almacenamiento y otro bajo el plato. A estos se suma el módulo DS3231, que provee la referencia temporal necesaria para la gestión autónoma de los horarios de dispensado.

El bloque de control concentra toda la lógica de la aplicación en el microcontrolador, e incorpora también el motor paso a paso NEMA 17 con su driver A4988, el pulsador de dispensado manual y el LED de estado del sistema.

El bloque de comunicación con el usuario está compuesto por el módulo ESP32, que opera como puente entre el microcontrolador y la aplicación de escritorio desarrollada en Flutter, a través de Bluetooth Classic.

3.2. Arquitectura de hardware

En esta sección se describe la interconexión entre los componentes electrónicos que conforman el sistema, junto con el análisis de potencia necesario para garantizar su correcto funcionamiento.

3.2.1. Gestión de alimentación

En primera instancia, se debe resolver el requerimiento de alimentación del sistema. A diferencia de soluciones portátiles basadas en batería, el presente trabajo utiliza una fuente de alimentación de 12 V de corriente alterna como fuente de energía principal. Esta decisión se fundamenta en el elevado consumo del motor paso a paso NEMA 17 con su driver A4988, que en condiciones de máxima carga puede demandar hasta 2 A, lo que hace inviable el uso de una batería para una operación continua y prolongada.

Dado que la totalidad de los componentes del sistema opera a una tensión nominal de 5 V, se incorporó un regulador de voltaje reductor (step-down) basado en el integrado LM2596S [22], capaz de aceptar tensiones de entrada de hasta 35 V y entregar una corriente máxima de 3 A con tensión de salida ajustable. La figura 3.2 ilustra su conexión con el microcontrolador.

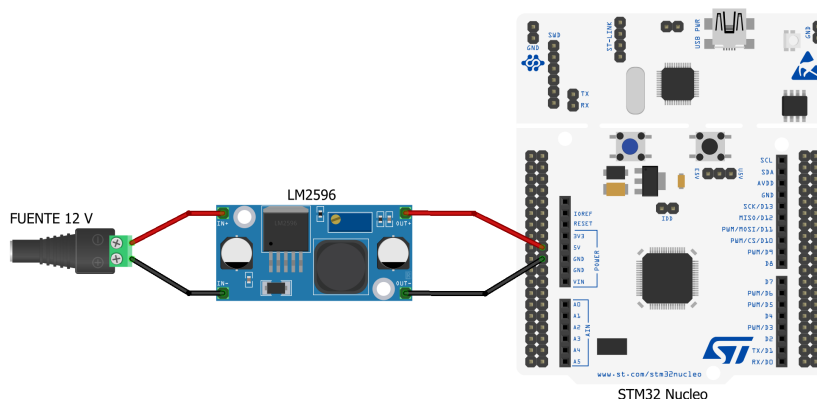


FIGURA 3.2. Interconexión entre la fuente de alimentación y el microcontrolador.

3.2.2. Interconexión de componentes

Una vez presentados los componentes del sistema en el capítulo anterior, se describe a continuación cómo se interconectan entre sí. Todos los módulos operan a una tensión nominal de 5 V, por lo que comparten la misma fuente de alimentación.

El microcontrolador STM32F446RE se comunica con los dos módulos HX711 mediante señales digitales GPIO. Cada módulo HX711 emplea una interfaz de dos cables: uno de reloj (CLK) y uno de datos (DATA), conectados a pines GPIO del microcontrolador configurados como salida y entrada respectivamente. El módulo DS3231 se conecta al microcontrolador a través del bus I²C, utilizando las líneas SCL y SDA del periférico I2C1, configurado a 400 kHz. El módulo ESP32 se conecta mediante el periférico UART4, configurado a 115.200 baudios con formato 8N1. El pulsador de dispensado manual se conecta a un pin configurado como entrada con interrupción externa (EXTI), con un tiempo de antirrebote de 200 ms implementado por software. El LED de estado se conecta a una salida GPIO de propósito general. La figura 3.3 muestra el esquema de estas conexiones.

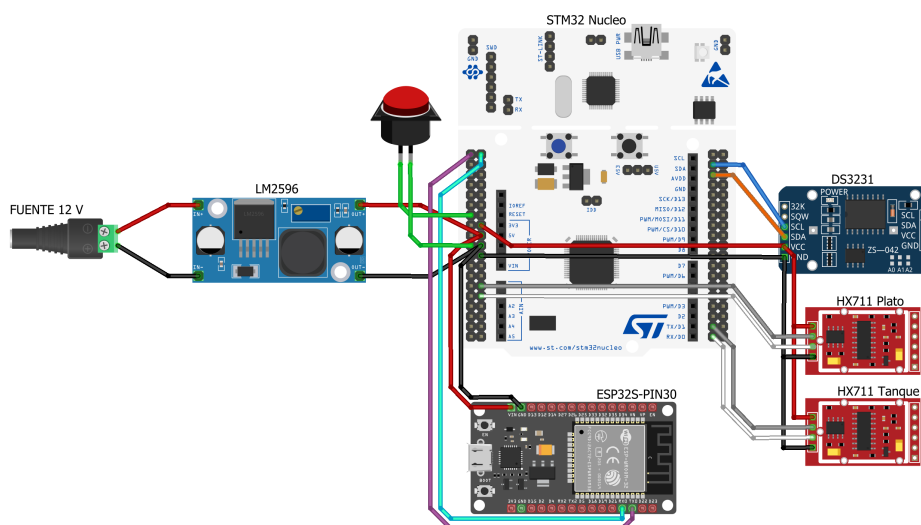


FIGURA 3.3. Interconexión entre el microcontrolador y sus periféricos.

El motor paso a paso NEMA 17 se controla a través del driver A4988, que recibe tres señales digitales provenientes del microcontrolador. La señal `STEP`, generada por el canal 3 del temporizador `TIM2` en modo PWM sobre el pin `PB10`, determina la velocidad y el número de pasos ejecutados por el motor. La señal `DIR` define el sentido de giro, mientras que la señal `ENABLE` habilita o deshabilita el driver. Las señales `DIR` y `ENABLE` se manejan como salidas GPIO simples. La figura 3.4 ilustra el esquema de conexión entre el microcontrolador, el driver y el motor.

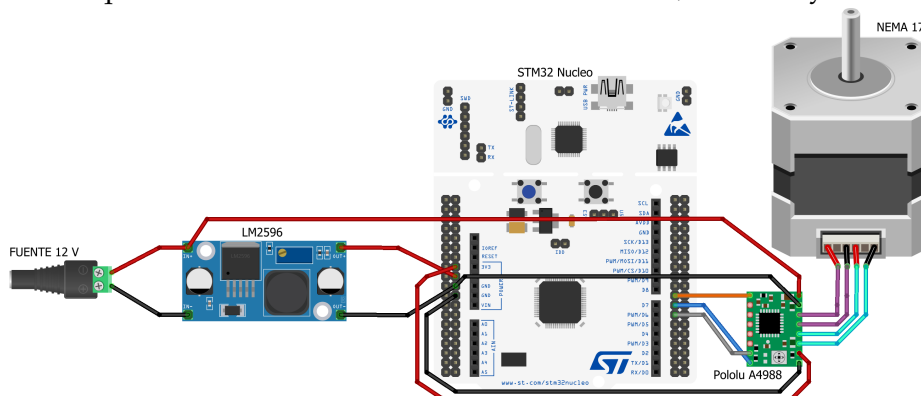


FIGURA 3.4. Interconexión entre el microcontrolador, el driver A4988 y el motor NEMA 17.

3.2.3. Análisis de potencia

El prototipo desarrollado requiere una fuente de alimentación capaz de proveer la corriente suficiente para todos los componentes del sistema de forma simultánea. La tabla 3.1 resume el consumo máximo de cada componente según las hojas de datos de sus fabricantes.

TABLA 3.1. Consumo máximo de cada componente del sistema.

Componente	Tensión (V)	Corriente máxima (mA)
STM32F446RE	5	36
HX711 (×2)	5	3
DS3231	5	1
Driver A4988	5	10
Motor NEMA 17	12	1700
ESP32	5	130
LED de estado	5	20

Del análisis de la tabla se concluye que el componente de mayor consumo es el motor paso a paso NEMA 17, que opera a 12 V y puede demandar hasta 1,7 A por fase en condiciones de máxima carga. El resto de los componentes opera a 5 V con un consumo total aproximado de 200 mA. Estos valores justifican la elección de una fuente de alimentación de 12 V y el módulo regulador LM2596S, cuya corriente máxima de salida de 3 A provee el margen necesario para una operación confiable del sistema.

3.3. Diseño del firmware

En esta sección se describe la organización del firmware desarrollado para el microcontrolador STM32F446RE y la lógica de control que gobierna el comportamiento del sistema.

3.3.1. Organización del firmware

Para implementar la lógica del trabajo, se desarrollaron módulos de software de forma independiente, con el propósito de que su desarrollo fuera autónomo y para facilitar su integración. La figura 3.5 muestra el diagrama en bloques que representa los subsistemas que conforman el firmware.

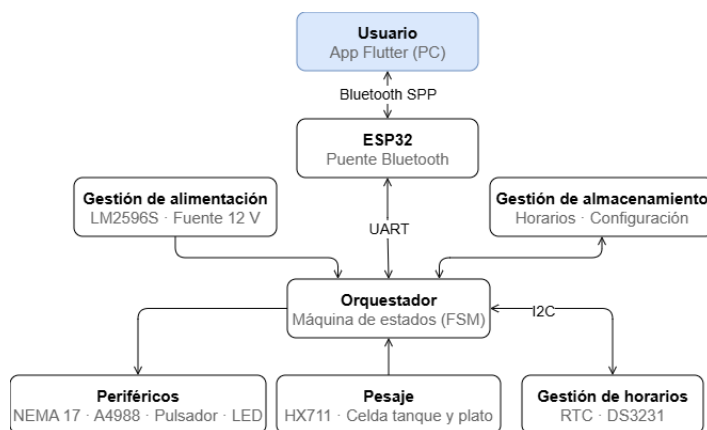


FIGURA 3.5. Diagrama en bloques de los subsistemas de software.

El módulo central del sistema es el orquestador, cuya función es inicializar todos los periféricos necesarios y luego gestionar la lógica de control a través de una máquina de estados. Para ello se utilizó FreeRTOS [8], con dos tareas principales: una encargada del control del dispensado y otra dedicada a la comunicación con el usuario.

3.3.2. Módulo orquestador

El módulo orquestador gestiona el ciclo completo de operación del dispensador mediante una máquina de estados finita. La figura 3.6 presenta el diagrama de estados del sistema.

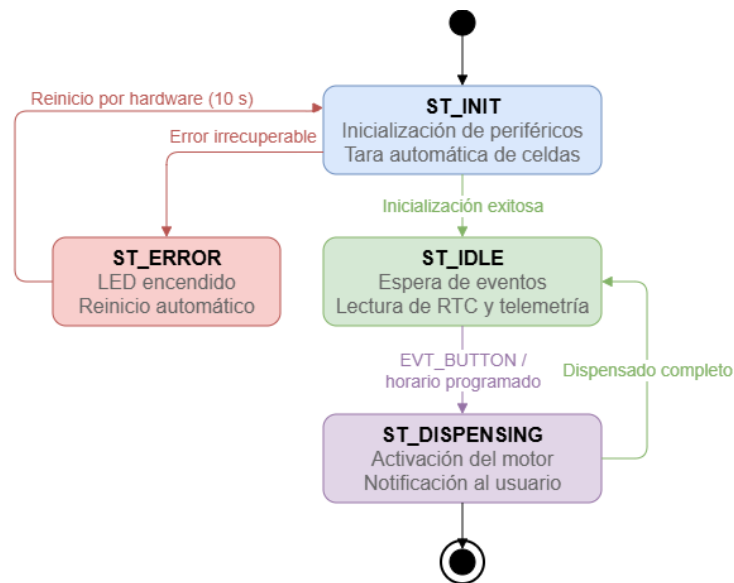


FIGURA 3.6. Diagrama de estados del dispensador.

El estado `ST_INIT` corresponde a la inicialización del sistema: se configuran los periféricos, se establece la comunicación con los módulos HX711 y DS3231, y se ejecuta la tara automática de las celdas de carga. Si la inicialización concluye de forma exitosa, el sistema transita al estado `ST_IDLE`. Ante un error irrecuperable, transita al estado `ST_ERROR`, donde el LED permanece encendido y el sistema realiza un reinicio automático tras 10 s.

En el estado `ST_IDLE`, el sistema aguarda eventos: lee periódicamente el reloj de tiempo real, transmite la telemetría hacia la aplicación y evalúa la ocurrencia de un horario programado o de una pulsación manual. Cuando se detecta el evento `EVT_BUTTON` o `EVT_SCHEDULED_FEED`, el sistema transita al estado `ST_DISPENSING`, donde activa el motor, notifica al usuario y aguarda la confirmación del ciclo. Al completarse el dispensado, retorna a `ST_IDLE`.

3.3.3. Módulo de periféricos

El módulo de periféricos agrupa los componentes de actuación y señalización del sistema: el motor paso a paso NEMA 17, el pulsador de dispensado manual y el LED de estado.

El control del motor se implementó mediante el driver A4988. La señal `STEP` se genera por el canal 3 del temporizador `TIM2` en modo PWM, y su frecuencia

determina la velocidad de rotación. Las señales `DIR` y `ENABLE`, que controlan el sentido de giro y la habilitación del driver respectivamente, se implementaron como salidas GPIO de propósito general. La cantidad de pasos ejecutados en cada ciclo de dispensado determina la porción de alimento entregada.

El pulsador de dispensado manual se conectó a un pin configurado con interrupción externa (`EXTI`). La rutina de servicio de interrupción implementó un mecanismo de antirrebote de 200 ms por software y publicó un evento en la cola de despacho del sistema operativo para notificar a la tarea de control del dispensado.

El LED de estado refleja el estado activo de la máquina de estados: permanece encendido de forma continua en `ST_ERROR`, parpadea a 1 Hz durante `ST_IDLE` y se apaga en `ST_DISPENSING`.

3.3.4. Módulo de pesaje

El módulo de pesaje gestiona la inicialización y la lectura periódica de los dos módulos HX711 conectados a las celdas de carga del tanque y del plato.

Durante el estado `ST_INIT`, el módulo ejecuta una tara automática que establece el valor de referencia de reposo de cada celda. A partir de ese punto, las lecturas de peso se obtienen a partir de un factor de calibración aplicado sobre los valores digitales de 24 bits provistos por el HX711, y se expresan en gramos. Para reducir el ruido en la señal, cada lectura resulta del promedio de 8 muestras consecutivas.

3.3.5. Módulo de gestión de horarios

El módulo de gestión de horarios compara periódicamente la hora provista por el módulo DS3231 con la lista de horarios de dispensado almacenados. En cada ciclo de la tarea de control, durante el estado `ST_IDLE`, se consultan las horas y los minutos del reloj de tiempo real y se verifica si alguno de los horarios activos coincide con el instante actual. Cuando se detecta una coincidencia, se genera el evento `EVT_SCHEDULED_FEED`, que desencadena la transición al estado `ST_DISPENSING`. Un mecanismo de inhibición evita que el mismo horario dispare múltiples eventos dentro de la misma ventana temporal de un minuto.

3.3.6. Módulo de gestión de almacenamiento

El módulo de gestión de almacenamiento tiene como función conservar la configuración del sistema a través de los ciclos de encendido y apagado. Los horarios de dispensado se almacenan en la memoria flash interna del microcontrolador STM32F446RE, lo que permite recuperarlos ante un eventual reinicio del sistema.

El sistema admite hasta diez horarios programados. Cada entrada incluye la hora (0–23), los minutos (0–59), un indicador de habilitación y una etiqueta de texto identificatoria. Al iniciarse el sistema, los datos se cargan desde la flash hacia una estructura en memoria RAM, donde son consultados por el módulo de gestión de horarios durante la operación normal.

La actualización de los horarios se origina en la aplicación de escritorio, que envía el comando `set_schedules` al sistema a través del protocolo de comunicación descrito en la sección 3.4.1. Al recibir este comando, el módulo reemplaza el contenido previo en flash con los nuevos valores.

3.4. Comunicaciones e interfaces

En esta sección se describe la implementación del protocolo de comunicación entre el microcontrolador y la aplicación de escritorio, así como las funcionalidades y la estructura de navegación de la interfaz de usuario.

3.4.1. Módulo de comunicación

El módulo de comunicación gestiona el intercambio de información entre el microcontrolador y la aplicación de escritorio a través del módulo ESP32. La comunicación es bidireccional y se estructura en dos tipos de mensajes: los mensajes salientes, que el sistema envía hacia la aplicación, y los comandos entrantes, que la aplicación envía hacia el sistema. Todos los mensajes se estructuran en formato JSON [23], lo que permite una integración sencilla con la aplicación de escritorio. La figura 3.7 ilustra la arquitectura de comunicación del sistema junto con los mensajes intercambiados.

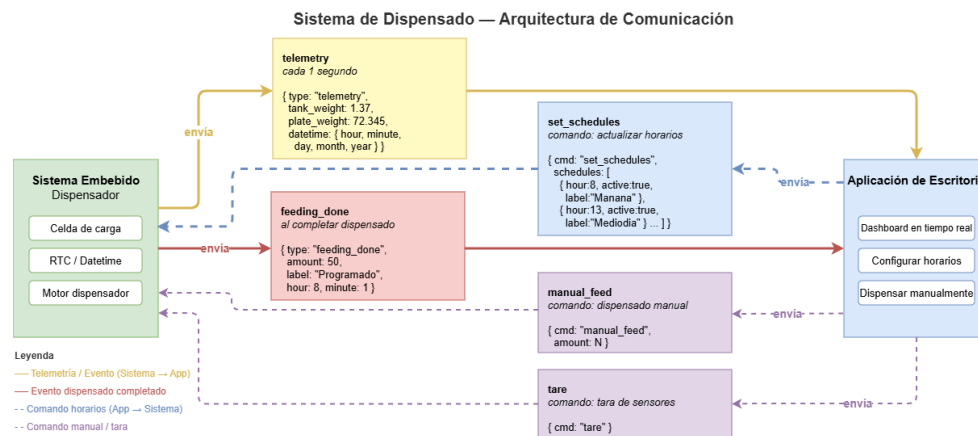


FIGURA 3.7. Arquitectura de comunicación del sistema de dispensado.

El sistema transmite dos tipos de mensajes hacia la aplicación. El primero es un mensaje de telemetría transmitido cada 1 segundo que incluye el peso del tanque de almacenamiento, el peso del plato y la fecha y hora provistas por el módulo DS3231. Al completarse un ciclo de dispensado, el sistema emite un mensaje de evento con la cantidad dispensada, la etiqueta del tipo de dispensado y la hora en que ocurrió. Entre los comandos entrantes, la aplicación puede solicitar un dispensado manual mediante `manual_feed`, ejecutar la tara de las celdas de carga mediante `tare`, o actualizar los horarios programados mediante `set_schedules`.

3.4.2. Interfaz de usuario

La interfaz de usuario se implementó como una aplicación de escritorio desarrollada en Flutter, que corre sobre el sistema operativo Windows y se comunica con el sistema mediante Bluetooth Classic. La aplicación permite al usuario visualizar en tiempo real el peso del tanque de almacenamiento y del plato de consumo, consultar el historial de dispensados, configurar los horarios programados y ejecutar un dispensado manual o una tara de las celdas de carga.

La aplicación se organiza en cinco pantallas principales accesibles desde una barra de navegación inferior: Inicio, Horarios, Historial, Vincular y Ajustes. La figura 3.8 presenta el diagrama de navegación de la aplicación con el contenido de cada pantalla.

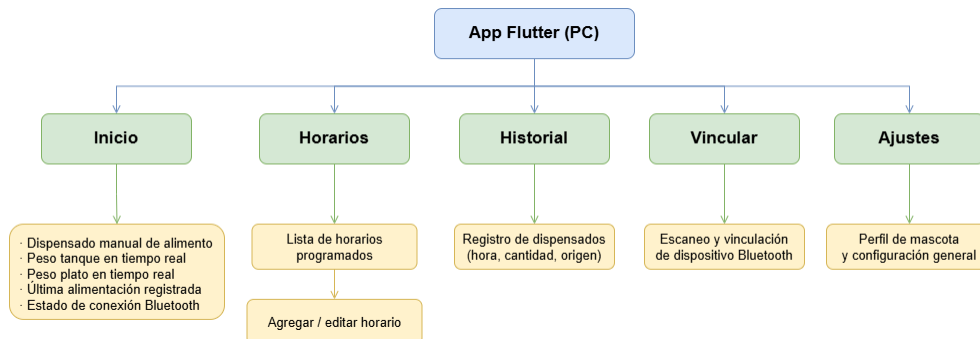


FIGURA 3.8. Diagrama de navegación de la aplicación de escritorio.

La pantalla de inicio muestra el *dashboard* principal con los valores de peso actualizados en tiempo real y el estado de la última alimentación registrada. La pantalla de horarios permite consultar y gestionar los horarios programados. La pantalla de historial presenta el registro completo de eventos de dispensado con fecha, hora y cantidad. La pantalla de vinculación facilita el proceso de conexión Bluetooth con el dispositivo. Finalmente, la pantalla de ajustes permite configurar el perfil de la mascota y los parámetros generales de la aplicación.

3.5. Integración hardware-software

En esta sección se describe cómo los módulos de hardware y firmware interactúan durante la operación del sistema. La figura 3.9 ilustra el diagrama de integración, donde se observa la relación entre las tareas de FreeRTOS y los componentes físicos del sistema.

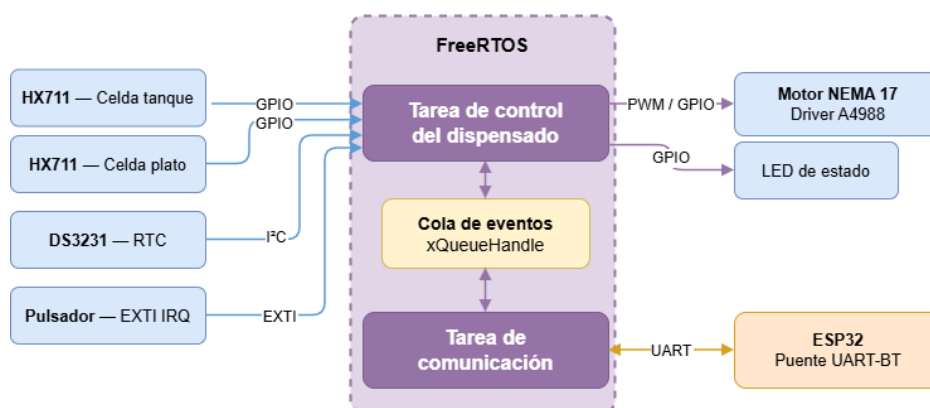


FIGURA 3.9. Diagrama de integración hardware-software del sistema.

El firmware se estructuró en torno a dos tareas concurrentes de FreeRTOS: la tarea de control del dispensado y la tarea de comunicación. La primera gestiona

directamente los periféricos de hardware: lee el peso de las celdas de carga a través de los módulos HX711, consulta la hora al módulo DS3231, controla el motor NEMA 17 y atiende las interrupciones del pulsador. La segunda se encarga exclusivamente del intercambio de mensajes con la aplicación de escritorio a través del módulo ESP32 por UART.

La sincronización entre ambas tareas se realiza mediante una cola de mensajes de FreeRTOS (`xQueueHandle`). Cuando la tarea de comunicación recibe un comando proveniente de la aplicación, lo publica en la cola. La tarea de control del dispensado consume estos eventos en cada iteración de la máquina de estados y ejecuta la acción correspondiente. Al completarse un ciclo de dispensado, la tarea de control publica el resultado en la cola para que la tarea de comunicación lo notifique a la aplicación. Este mecanismo garantiza que el procesamiento de comandos externos no interfiera con el ciclo de control del hardware, y ambas tareas operen de forma desacoplada.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] A. W. Rollins y M. Murphy. «Nutritional assessment in the cat». En: *Journal of Feline Medicine and Surgery* (abr. de 2019), págs. 442-448. URL: <https://journals.sagepub.com/doi/10.1177/1098612X19843213>.
- [2] Julie A. Churchill y Laura Eirmann. «Senior Pet Nutrition and Management». En: *Veterinary Clinics of North America: Small Animal Practice* 51.3 (mayo de 2021). Epub 2021 Feb 27. PMID: 33653535, págs. 635-651. URL: <https://doi.org/10.1016/j.cvsm.2021.01.004>.
- [3] Hikmat Yar et al. «Towards Smart Home Automation Using IoT-Enabled Edge-Computing Paradigm». En: *Sensors* 21.14 (2021), pág. 4932. URL: <https://doi.org/10.3390/s21144932>.
- [4] PetSafe. *Alimentador de mascotas automático e inteligente Smart Feed*. Consultado: mayo 2026. 2026. URL: <https://www.petsafe.com/es/p/alimentador-de-mascotas-automatico-e-inteligente-smart-feed/PFD19-16861/>.
- [5] PetSafe. *My PetSafe*. Aplicación móvil. iOS: <https://apps.apple.com/us/app/my-petsafe/id1498476188>. Android: <https://play.google.com/store/apps/details?id=net.petsafe.platform>. 2026.
- [6] PETLIBRO. *Granary 5G Wi-Fi Automatic Pet Feeder*. Consultado: mayo 2026. 2026. URL: <https://petlibro.com/products/petlibro-5g-wifi-automatic-pet-feeder>.
- [7] AVIA Semiconductor. *HX711: 24-Bit Analog-to-Digital Converter for Weigh Scales*. Hoja de datos. 2016. URL: https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf.
- [8] Amazon Web Services. *FreeRTOS Reference Manual, Version 10.0.0*. Manual de referencia oficial. 2017. URL: https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf.
- [9] STMicroelectronics. *STM32F446xC/E: Arm® Cortex®-M4 32-bit MCU+FPU, 225 DMIPS, up to 512 kB Flash/128+4 KB RAM*. Hoja de datos. Ene. de 2021. URL: <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>.
- [10] Texas Instruments. *UART: A Hardware Communication Protocol*. Nota de aplicación. 2020. URL: <https://www.ti.com/lit/ug/sprugp1/sprugp1.pdf>.
- [11] NXP Semiconductors. *I2C-Bus Specification and User Manual*. Especificación técnica. 2021. URL: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>.
- [12] STMicroelectronics. *NUCLEO-F446RE: STM32 Nucleo-64 Development Board*. Placa de evaluación. 2024. URL: <https://www.st.com/en/evaluation-tools/nucleo-f446re.html>.
- [13] Espressif Systems. *ESP32 Series Datasheet*. Hoja de datos. 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [14] Pololu Corporation. *Stepper Motor: Bipolar, 200 Steps/Rev, 42×48mm, 2.8V, 1.7A/Phase*. Hoja de datos. 2023. URL: <https://www.pololu.com/product/1200>.

- [15] Allegro MicroSystems. *A4988 DMOS Microstepping Driver with Translator and Overcurrent Protection*. Hoja de datos. 2014. URL: <https://www.allegromicro.com/~media/Files/Datasheets/A4988-Datasheet.ashx>.
- [16] Analog Devices. *DS3231: Extremely Accurate I2C-Integrated RTC/TCXO/Crystal*. Hoja de datos. 2015. URL: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>.
- [17] STMicroelectronics. *STM32CubeIDE Integrated Development Environment*. Entorno de desarrollo integrado. 2024. URL: <https://www.st.com/en/development-tools/stm32cubeide.html>.
- [18] Doxygen. *Doxygen: Generate Documentation from Source Code*. Herramienta de documentación de código fuente. 2024. URL: <https://www.doxygen.nl>.
- [19] PlatformIO Labs. *PlatformIO: A Professional Collaborative Platform for Embedded Development*. Entorno de desarrollo integrado. 2024. URL: <https://platformio.org>.
- [20] Google LLC. *Flutter: Build Apps for Any Screen*. Framework de desarrollo de aplicaciones multiplataforma. 2024. URL: <https://flutter.dev>.
- [21] Bluetooth SIG. *Serial Port Profile (SPP)*. Consultado: mayo 2026. 2003. URL: <https://www.bluetooth.com/specifications/specs/serial-port-profile-1-1/>.
- [22] Texas Instruments. *LM2596 Series: SIMPLE SWITCHER Power Converter 150 kHz 3A Step-Down Voltage Regulator*. 2020. URL: <https://www.ti.com/lit/ds/symlink/lm2596.pdf>.
- [23] JSON.org. *Introducing JSON*. Consultado: mayo 2026. 2023. URL: <https://www.json.org/json-en.html>.